

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Manual 14

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Evaluation metrics for Regression	3
1.1 Mean Absolute Error (MAE)	3
1.2 Mean Squared Error (MSE)	4
1.3 Root Mean Squared Error (RMSE)	4
1.4 R-squared (R^2) Score	5
2 Evaluation metrics for Classification	5
2.1 Confusion Matrix	7
2.2 Accuracy	8
2.3 Precision	9
2.4 Recall	9
2.5 F1 Score	9
2.6 Area Under Curve (AUC) and ROC Curve	10
2.7 Classification Report	11

Objectives

After performing this lab, students shall be able to:

- ✓ Implement Evaluation metrics for regression - Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared (R^2), and Adjusted R-squared from scratch.
- ✓ Validate correctness by comparing results with Scikit-learn's metric functions.
- ✓ Manually compute and interpret evaluation metrics for classification:
 - Confusion Matrix
 - Precision
 - Recall
 - F1-score
- ✓ Verify manually computed results using Scikit-learn's built-in methods.

1 Evaluation metrics for Regression

Regression models predict continuous outcomes (e.g., house prices, temperatures). To measure their performance, we need evaluation metrics that quantify how close predictions are to actual values. There is no single “best” metric; each has strengths and weaknesses.

1.1 Mean Absolute Error (MAE)

In the fields of statistics and machine learning, the Mean Absolute Error (MAE) is a frequently employed metric. It's a measurement of the typical absolute discrepancies between a dataset's actual values and projected values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |x_i - y_i|$$

Where:

- x_i represents the actual or observed values for the i -th data point.
- y_i represents the predicted value for the i -th data point.

```
from sklearn.metrics import mean_absolute_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mae = mean_absolute_error(true_values, predicted_values)
print("Mean Absolute Error:", mae)
```

1.2 Mean Squared Error (MSE)

A popular metric in statistics and machine learning is the Mean Squared Error (MSE). It measures the square root of the average discrepancies between a dataset's actual values and projected values. MSE is frequently utilized in regression issues and is used to assess how well predictive models work.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2$$

Where:

- x_i represents the actual or observed values for the i -th data point.
- y_i represents the predicted value for the i -th data point.

```
from sklearn.metrics import mean_squared_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

mse = mean_squared_error(true_values, predicted_values)
print("Mean Squared Error:", mse)
```

1.3 Root Mean Squared Error (RMSE)

RMSE stands for Root Mean Squared Error. It is a usually used metric in regression analysis and machine learning to measure the accuracy or goodness of fit of a predictive model, especially when the predictions are continuous numerical values.

$$RMSE = \sqrt{MSE}$$

```
from sklearn.metrics import mean_squared_error

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

rmse = np.sqrt(mean_squared_error(true_values, predicted_values))
print("Root mean Squared Error:", rmse)
```

1.4 R-squared (R^2) Score

A statistical metric frequently used to assess the goodness of fit of a regression model is the R-squared (R^2) score, also referred to as the coefficient of determination. It quantifies the percentage of the dependent variable's variation that the model's independent variables contribute to. R^2 is a useful statistic for evaluating the overall effectiveness and explanatory power of a regression model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

```
from sklearn.metrics import r2_score

true_values = [2.5, 3.7, 1.8, 4.0, 5.2]
predicted_values = [2.1, 3.9, 1.7, 3.8, 5.0]

r2 = r2_score(true_values, predicted_values)
print("R-squared ( $R^2$ ) Score:", r2)
```

2 Evaluation metrics for Classification

We will be using the following sample dataset for the given example code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score,
    classification_report,
    roc_curve
)

# Example true labels and predicted labels
```

```

y_true = [0, 1, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 0, 0, 0, 1, 1, 1]
y_pred_proba = [0.1, 0.9, 0.2, 0.4, 0.3, 0.8, 0.7, 0.6] # Probabilities
for ROC-AUC

```

2.1 Confusion Matrix

Confusion matrix creates a N X N matrix, where N is the number of classes or categories that are to be predicted.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

		Actual Value	
		Positive	Negative
Predicated Value	Positive	5600	600
	Negative	500	3300

- True Positives: Correctly predicted positive class.
- True Negatives: Correctly predicted negative class.
- False Positives: Incorrectly predicted positive class (Type I error)
- False Negatives: Incorrectly predicted negative class (Type II error)

```

cm = confusion_matrix(y_true, y_pred)
print("\nConfusion Matrix:\n", cm)

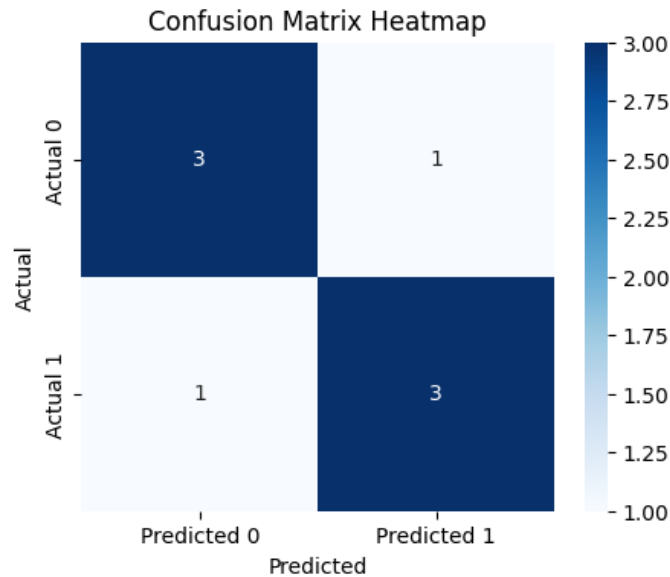
```

To visualize confusion matrix:

```

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual
0', 'Actual 1'])
plt.title('Confusion Matrix Heatmap')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

```



2.2 Accuracy

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

While accuracy provides a quick snapshot, it can be misleading in cases of imbalanced datasets. For example, in a dataset with 90% class A and 10% class B, a model predicting only class A will still achieve 90% accuracy but it will fail to identify any class B instances.

Accuracy is good but it gives a False Positive sense of achieving high accuracy. The problem arises due to the possibility of misclassification of minor class samples being very high.

```
accuracy = accuracy_score(y_true, y_pred)
print("Accuracy:", accuracy)
```

2.3 Precision

It measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision helps ensure that when the model predicts a positive outcome, it's likely to be correct.

```
precision = precision_score(y_true, y_pred)
print("Precision:", precision)
```

2.4 Recall

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

In scenarios where catching all positive cases is important (like disease detection), recall is a key metric.

```
recall = recall_score(y_true, y_pred)
print("Recall:", recall)
```

2.5 F1 Score

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision gives us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
f1 = f1_score(y_true, y_pred)
print("F1 Score:", f1)
```


2.6 Area Under Curve (AUC) and ROC Curve

It is useful for binary classification tasks. The AUC value represents the probability that the model will rank a randomly chosen positive example higher than a randomly chosen negative example. AUC ranges from 0 to 1 with higher values showing better model performance.

ROC Curve is a graphical representation of the True Positive Rate (TPR) vs the False Positive Rate (FPR) at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (TPR) and specificity (1 - FPR) across various thresholds. Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

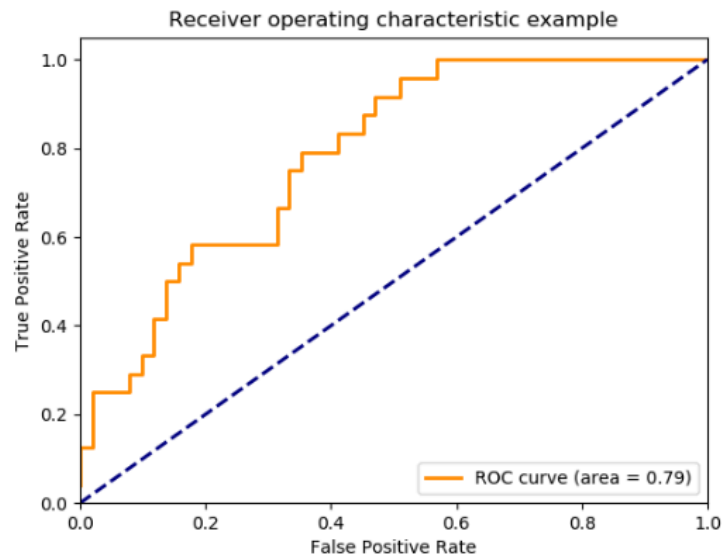
$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).

```
roc_auc = roc_auc_score(y_true, y_pred_proba)
print("ROC-AUC Score:", roc_auc)
```

ROC visualization:

```
fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)
plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, color='blue', label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0,1], [0,1], color='red', linestyle='--')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```



2.7 Classification Report

A classification report is a summary table that shows how well your classification model performed, by giving key metrics Precision, Recall, F1-score, and Support for each class in your data.

```
report = classification_report(y_true, y_pred)
print("\nClassification Report:\n", report)
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.75         0.75         0.75         4
     1       0.75         0.75         0.75         4

 accuracy          0.75
 macro avg         0.75         0.75         0.75         8
 weighted avg      0.75         0.75         0.75         8
```

Reference:

<https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>